

Embedded insights

Nathan Warner



**Northern Illinois
University**

Computer Science
Northern Illinois University
United States

Contents

1	Concepts	2
1.1	MCU Architecture	7
2	Context	12
2.1	Documents	12
2.2	Packages	12
2.3	STM32CubeMX	12
2.4	Generating a Project via CLI	13
2.5	The System	13
2.6	Basics	13
2.7	Edit, Build, Flash, Debug cycle	14

Concepts

- **Electronics**
 - Voltage, current, resistance, and power
 - Ohm's law
 - Digital high and low voltage levels
 - GPIO input versus output
 - Pull-up and pull-down resistors
 - LEDs and current-limiting resistors
 - Push buttons and switch bouncing
 - Capacitors and basic RC circuits
 - Analog versus digital signals
 - Reading schematics and pinout diagrams
 - Common ground and safe voltage limits
 - Basic use of a multimeter and logic analyzer
- **MCU Architecture:** Learn how a microcontroller differs from a desktop processor:
 - CPU core, flash, SRAM, and peripherals
 - Memory-mapped I/O
 - Registers and special-function registers
 - Cortex-M4 register set
 - Vector table
 - Main stack pointer
 - Reset sequence
 - Privileged and unprivileged execution
 - Exceptions and interrupts
 - NVIC interrupt controller
 - SysTick timer
 - Memory protection unit
 - Floating-point unit
 - Little-endian representation
 - Thumb and Thumb-2 instructions
- **The embedded build process**
 - Cross-compilation
 - Preprocessor, compiler, assembler, and linker stages
 - Object files and ELF files
 - Sections such as .text, .rodata, .data, and .bss
 - Symbols and relocation
 - Linker scripts
 - Startup assembly
 - Vector-table placement

- Copying .data from flash to SRAM
- Zero-initializing .bss
- Map files
- Converting ELF to BIN or Intel HEX
- Flashing versus debugging

- **Essential STM32 peripherals:**

1. GPIO
2. Clock system and RCC
3. External interrupts
4. General-purpose timers
5. PWM
6. UART
7. ADC
8. SPI
9. I²C
10. DMA
11. Watchdogs
12. RTC and low-power modes

For every peripheral, understand:

- Which clock enables it
- Which GPIO alternate function it uses
- Which registers configure it
- How status flags work
- How polling differs from interrupts
- How DMA can move data without continuous CPU involvement
- What errors and edge cases can occur

Board exercises include LED blinking with a timer, button interrupts, UART command input, potentiometer sampling through the ADC, PWM brightness control, and reading a sensor through I²C or SPI.

- **Interrupts and concurrency:**

- Interrupt service routines
- Interrupt latency
- Interrupt priorities
- Shared state between main code and an ISR
- Atomic operations and critical sections
- Race conditions
- Reentrancy
- Keeping ISRs short
- Deferred processing
- Ring buffers

- Producer-consumer design
- Why blocking inside an ISR is dangerous
- Why volatile alone does not solve concurrency

Board exercise: receive UART bytes in an interrupt, place them into a ring buffer, and process them from the main loop.

- **Time and real-time behavior:**

- Busy waiting versus hardware timers
- Blocking versus nonblocking code
- Deadlines, latency, jitter, and throughput
- Periodic and event-driven tasks
- Finite-state machines
- Cooperative scheduling
- Monotonic timestamps
- Timer overflow
- Input capture and output compare
- Debouncing without blocking delays

Board exercise: run LED control, button handling, UART processing, and sensor sampling concurrently without calling a long blocking delay.

- **Hardware communication protocols:**

- **UART** Baud rate, frames, parity, buffering, overruns
- **SPI** Controller/peripheral roles, chip select, CPOL, CPHA, full duplex
- **I²C** Addresses, START/STOP, ACK/NACK, pull-ups, clock stretching
- **CAN** Frames, arbitration, identifiers, error handling
- **USB** Endpoints, enumeration, descriptors; learn later

- **Drivers and firmware architecture:** Learn to separate:

- Application logic
- Hardware-independent modules
- Device drivers
- Board support code
- MCU peripheral access
- Vendor HAL or low-level libraries

Also understand:

- Hardware abstraction layers
- Dependency injection through function pointers
- State machines
- Event queues
- Error reporting
- Timeouts
- Configuration versus runtime state

- Interfaces suitable for host-based testing
- **RTOS concept:** Do not begin with an RTOS. First become comfortable with interrupts, timers, state machines, and a superloop. Then learn
 - Tasks and context switching
 - Preemptive scheduling
 - Priorities
 - Queues
 - Semaphores
 - Mutexes
 - Software timers
 - Event groups
 - Priority inversion
 - Deadlock
 - ISR-safe APIs
 - Per-task stack sizing
 - Heap-allocation strategies
 - Tick frequency and tickless operation
- **For employment, you should at least recognize::**
 - Watchdog design
 - Brownout and reset handling
 - Defensive timeouts
 - Fault recovery
 - Firmware versioning
 - Bootloaders and firmware updates
 - CRCs and data integrity
 - Power loss during flash writes
 - Static analysis
 - Coding standards such as MISRA C
 - Unit, integration, and hardware-in-the-loop testing
 - Resource limits
 - Low-power design
 - Security fundamentals
- **Progression:**

Stage	Board work	Main concepts
1	LED and button	GPIO, C bit operations, schematics
2	Build outside the IDE	GCC, ELF, linker script, startup code
3	Timer-driven LED	clocks, timers, interrupts, NVIC
4	UART command console	serial communication, ring buffers
5	ADC and PWM	analog input, sampling, timer output
6	I ² C/SPI	sensor protocols, datasheets, reusable drivers
7	DMA data acquisition	concurrency, buffering, throughput
8	Cooperative application	state machines, events, nonblocking design
9	FreeRTOS application	tasks, queues, synchronization
10	Portfolio project	architecture, testing, fault handling

- **For job**

- Write and debug C confidently.
- Explain startup, linking, flash, SRAM, and the vector table.
- Configure basic peripherals from their documentation.
- Write interrupt-driven, nonblocking firmware.
- Use UART, SPI, I²C, timers, ADC, PWM, and DMA.
- Diagnose faults using GDB and hardware tools.
- Explain races, critical sections, queues, and RTOS scheduling.
- Structure firmware into testable drivers and application modules.
- Present one substantial board project with documentation and tests.

1.1 MCU Architecture

- **Intro:** A Microcontroller Unit (MCU) architecture integrates a central processing unit, memory, and input/output peripherals onto a single integrated circuit
 - **CPU (Central Processing Unit):** The brain that fetches, decodes, and executes program instructions. It includes the Arithmetic Logic Unit (ALU) for math and logic, and the Control Unit for sequencing operations
 - **Program memory (ROM / flash):** Non-volatile storage that holds your permanent application code so it is not lost when power turns off.
 - **Data Memory (RAM):** Volatile storage used by the CPU to temporarily save active variables and runtime data
 - **System Bus:** An internal highway of wires that lets the CPU talk to memory and hardware peripherals
 - **Peripherals:** Built-in hardware tools that let the chip interact with the physical world. Common examples include:
 - * **GPIO:** General-purpose input/output pins for turning things on or off.
 - * **ADC:** Analog-to-digital converters to read voltages from sensors.
 - * **Timers/Counters:** For measuring time, creating delays, or generating Pulse-Width Modulation (PWM) signals.
 - * **Communication Interfaces:** Protocols like UART, SPI, and I²C to talk to other chips
- **Von Neumann architecture:** Uses a single shared memory space and bus for both data and instructions. It is simple to design but creates a data bottleneck because the CPU cannot read an instruction and read/write data at the exact same time
- **Harvard architecture:** Uses physically separate memory blocks and separate buses for code and data. This lets the processor fetch an instruction while simultaneously accessing data, which speeds up real-time execution. Most modern MCUs use variations of this model
- **Execute-in-place (XiP):** instructions do not necessarily need to move from ROM to RAM before execution. Most modern microcontrollers execute code directly from flash memory (ROM) using a technique called Execute-in-Place (XiP).

In most microcontrollers (like ARM Cortex-M, AVR, or PIC), the CPU is wired to fetch instructions directly from the Flash/ROM bus.

Microcontrollers have very limited RAM (often only kilobytes) but larger Flash memory. Keeping the code in Flash saves precious RAM for data and variables.

Flash memory is generally slower than RAM. For ultra-high-speed operations, Flash access time can become a performance bottleneck.

If instructions are fetched from Flash while data is simultaneously read or written in RAM, that is a classic implementation of the Harvard architecture.

- **Flash memory:** Flash holds data without power. It normally contains:
 - The vector table
 - Machine instructions
 - Read-only constants
 - Initial values for initialized global variables

On the STM32F446, main flash begins at:


```
0 0x08 00 00 00
```

The processor normally executes instructions directly from flash. This is commonly called execute in place.

Flash is persistent but slower and more complicated to modify than SRAM. You cannot generally treat it as ordinary writable memory.

- **SRAM:** SRAM is volatile working memory. It loses its contents when power is removed.

It contains things such as:

- Global and static variables
- The stack
- The heap, if one is used
- Communication buffers
- Temporary computation results

The primary SRAM address range begins at:

```
0 0x20000000
```

Address range	Purpose
0x00000000...	Boot-memory alias
0x08000000–0x0807FFFF	512 KiB internal Flash
Mostly reserved/unmapped space	No physical memory responds here
Near 0x1FFF0000	Factory system memory and bootloader
0x20000000–0x2001FFFF	128 KiB SRAM

The address space is not one enormous physical memory device. Large address ranges are simply unused or reserved.

- **The boot-memory alias:** 0x00000000 is particularly important. At reset, the Cortex-M4 expects the vector table to appear there:

Address	Initial meaning
0x00000000	Initial Main Stack Pointer value
0x00000004	Address of the reset handler
0x00000008	Address of the NMI handler
0x0000000C	Address of the HardFault handler

- **Programs:** In ordinary embedded development, Flash contains one firmware image:

```
Flash
|-- Vector table
|-- Program instructions
|-- Read-only constants
|-- Initial values for global variables
```

That firmware might contain many modules, functions, tasks, or an RTOS, but collectively it is normally considered one program.

You can store multiple independent firmware images if you create a bootloader and partition Flash.

Region	Example size
Bootloader	32 KiB
Application A	200 KiB
Application B	200 KiB
Configuration/metadata	Remaining space

The bootloader would select an application, set the application's vector-table location, load its initial stack pointer, and jump to its reset handler.

Without a custom bootloader: normally one firmware program

- **Address space:** On the STM32F446RE, the address space is primarily a physical address space, not a virtual address space.

When the Cortex-M4 executes:

```
o LDR r0, [r1]
```

and r1 contains 0x20000100, that address is placed onto the MCU's internal bus. The bus interconnect routes it directly to the SRAM hardware.

Memory on the MCU is usually not virtual

- **Startup chain:**
 1. Reset happens
 2. boot mode selects memory
 3. CPU reads the vector table
 4. CPU enters the reset handler
 5. startup code calls main()
- **Resets:** A reset returns the processor and much of the microcontroller hardware to a known initial state. It does not erase the program stored in Flash.

A reset may be caused by:

- Power being applied
- The NRST reset pin being asserted
- A watchdog timeout
- Brownout or insufficient supply voltage
- Software requesting a reset
- A debugger pressing “reset”

Different reset sources affect slightly different parts of the MCU, but they generally restart program execution using the same boot process.

After reset:

- The CPU registers return to defined reset states.
 - Most peripherals return to their reset states.
 - The CPU uses the Main Stack Pointer, or MSP.
 - The CPU begins in privileged Thread mode.
 - Flash retains its contents.
 - Ordinary SRAM contents should be considered undefined, even if some electrical values happen to remain.
- **Bootting:** Booting is the process of deciding which code the processor should begin executing and then preparing that code to run.

For a normal embedded application, the process is:

1. Reset
2. Select boot memory
3. Read initial stack pointer
4. Read reset-handler address
5. Execute Reset_Handler
6. Initialize RAM and hardware
7. Call main()

The STM32F446 can boot from different physical memory regions. Boot configuration pins and option settings determine which region appears at address 0x00000000.

The primary choices are:

Boot source	Purpose
Main Flash	Run your normal firmware
System memory	Run ST’s factory-programmed bootloader
SRAM	Run a program previously loaded into SRAM

The Nucleo board normally holds BOOT0 low, selecting the main Flash.

Your program physically begins at:

```
1 0x08000000
```

During normal Flash boot, the STM32 aliases this memory at:

```
1 0x00000000
```

Both addresses refer to the same beginning of Flash during this boot configuration. The CPU reads the initial startup information from 0x00000000, while your linker normally places it physically at 0x08000000.

- **Vector table:**

Context

The STMicroelectronics NUCLEO-F446RE is a highly popular, mid-range development board belonging to the STM32 Nucleo-64 ecosystem. It is designed around the powerful STM32F446RET6 microcontroller

Arm Cortex-M4 core processor

2.1 Documents

- **UM1724 — STM32 Nucleo-64 board user manual:** Board connectors, jumpers, power options, LEDs, buttons, clock configuration, ST-LINK, and Arduino/Morpho pin mappings
- **DS10693 — STM32F446xC/E datasheet:** Pinouts, alternate pin functions, voltage/current limits, clock limits, memory capacity, and electrical characteristics
- **RM0390 — STM32F446xx reference manual:** Peripheral behavior and registers: GPIO, RCC, timers, UART, SPI, I²C, ADC, DMA, interrupts, flash, USB, and more
- **PM0214 — STM32 Cortex-M4 programming manual:** CPU registers, instruction set, exception handling, NVIC, SysTick, MPU, FPU, and processor operating modes
- **MB1136 board schematic:** Exact electrical connections between the STM32, LEDs, buttons, oscillators, ST-LINK, and headers
- **ES0298 — STM32F446 device errata:** Known hardware limitations and their workarounds
- **UM1725 — STM32F4 HAL and LL driver manual:** STM32Cube HAL and low-layer driver APIs

2.2 Packages

- arm-none-eabi-gcc Compiles C/C++ for the Cortex-M4
- arm-none-eabi-newlib Embedded C standard library
- arm-none-eabi-gdb Debugs the program running on the board
- openocd Communicates with the board's integrated ST-LINK

2.3 STM32CubeMX

STM32CubeMX is a graphical hardware-configuration and code-generation program made by STMicroelectronics. It helps you configure an STM32 microcontroller without manually writing all the startup and peripheral-initialization code.

2.4 Generating a Project via CLI

Create a new text file named extension `.script`. Then,

```
1  # 1. Set the target microcontroller (replace with your exact part
    ↪  number)
2  config load STM32F466RETx
```

it is highly recommended to load the board configuration instead of just the MCU. Loading the board pre-configures system peripherals like the default USART pins for the built-in ST-LINK debugger, the user LEDs, and push buttons.

```
1  board load NUCLEO-F446RE
2
3  # 2. Setup project path and name
4  project name <project name>
5  project path <path>
6
7  # 3. Choose your IDE/Toolchain (e.g., STM32CubeIDE, Makefile,
    ↪  EWARM, MDK-ARM)
8  project toolchain Makefile
9
10 # Saves the generated .ioc (input output configuration)
11 config saveas <path>
12
13 # 4. Generate the code and project structure
14 project generate
15
16 # 5. Exit the CLI tool
17 exit
```

```
1  stm32cubemx -q *.script
```

2.5 The System

- **Cortex-M4 core:** Executes instructions and contains the CPU registers, exception mechanism, and SysTick timer.
- **STM32F446RE MCU:** Combines the Cortex-M4 core with Flash, SRAM, GPIO, timers, UARTs, ADCs, and other peripherals.
- **NUCLEO-F446RE board:** Connects the MCU to an onboard LED, button, ST-LINK debugger, USB connector, headers, and supporting circuitry.

2.6 Basics

•

2.7 Edit, Build, Flash, Debug cycle

Now that we have the project files, open the `.ioc` and verify

- PA5 is configured as GPIO_Output.
- PA5 controls the green onboard LED, labeled LD2.
- SYS → Debug is set to Serial Wire.
- System Core → GPIO shows PA5 as an output.
- A project toolchain is selected under Project Manager.

In `main.c`, have

```
0  while (1)
1  {
2      HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
3      HAL_Delay(500);
4
5      /* USER CODE END WHILE */
6
7      /* USER CODE BEGIN 3 */
8  }
```

Then, build the project with the generated makefile.

- make compiles and links.
- OpenOCD communicates with the onboard ST-LINK.
- arm-none-eabi-gdb provides source-level debugging.

Use `Build/*.elf` for flashing and debugging. It contains:

- Program instructions and data.
- Flash addresses.
- Function and variable names.
- Source-line debugging information.

Check the result with:

```
1  arm-none-eabi-size build/*.elf
```

	text	data	bss	dec
	↪ hex	filename		
2	5544	20	1644	7208
	↪ 1c28	intro.elf		

Connect the USB cable to the ST-LINK USB connector and run:

```
1 openocd -f board/st_nucleo_f4.cfg
```

Successful output should mention:

```
1 STLINK
2 stm32f4x.cpu
3 Listening on port 3333 for gdb connections
```

The supplied `st_nucleo_f4.cfg` configures the ST-LINK interface, SWD transport, and STM32F4 target.

To flash the program, run

```
1 openocd \
2     -f board/st_nucleo_f4.cfg \
3     -c "program build/*.elf verify reset exit"
```

This performs four operations:

1. Erases the necessary Flash sectors.
2. Programs the .elf
3. Verifies the written data.
4. Resets and starts the MCU.

Your LED program should begin running immediately.

Debugging uses two terminals. In the first terminal, start OpenOCD:

```
1 openocd -f board/st_nucleo_f4.cfg
```

Leave it running. OpenOCD exposes a GDB server on TCP port 3333. In the second terminal, start GDB:

```
1 arm-none-eabi-gdb -tui build/intro.elf
```

At the GDB prompt, enter:

```
1 target extended-remote localhost:3333
2 monitor reset halt
3 load
4 monitor reset halt
5 break main
6 continue
```


- **Target:** is a command used to specify the execution environment of the program you are debugging. It connects your local GDB session (the host) to a process, core file, or a separate machine (the target).
- **load:** The load command in GDB is primarily used during remote debugging to transfer and upload an executable file directly onto a remote target system (such as an embedded microcontroller or a remote board)
- **monitor reset halt:** In GDB, the command monitor reset halt is used during embedded firmware debugging to reset the target hardware and immediately halt the CPU at the very first instruction

Monitor tells GDB to pass the trailing command directly to the underlying hardware debug server (such as OpenOCD or pyOCD) rather than executing it within GDB itself

Reset halt instructs the debug server to assert the hardware reset line and stop the processor core immediately upon release. This prevents the MCU from executing any application code or bootloaders, keeping it in a clean, predictable state

The first resets and halts the MCU so OpenOCD can program Flash while the processor is not executing code.

The second resets the MCU after programming so the newly loaded firmware begins with a clean hardware state:

- Stack pointer is reloaded.
- Program counter returns to Reset_Handler.
- Global initialization runs again.
- Peripherals return to their reset state.

The first reset is not always strictly necessary because OpenOCD can normally halt the target automatically during load.

The complete connection is

GDB → OpenOCD → onboard ST-LINK → SWD → STM32F446RE

build/intro.elf contains:

- ARM machine instructions.
- The addresses where sections belong in Flash and SRAM.
- The interrupt vector table.
- Initialized global data.
- Function and variable names.
- Source-line debugging information.
- Section metadata such as .text, .data, .bss, and .isr_vector.

The MCU never parses the ELF file. OpenOCD reads the ELF metadata, extracts its loadable machine-code sections, and writes them into the correct Flash locations. After reset, the Cortex-M4 fetches instructions directly from Flash.

You can inspect the ELF structure with:

```
1 arm-none-eabi-readelf -h build/*.elf
2 arm-none-eabi-objdump -h build/*.elf
3 arm-none-eabi-nm build/*.elf
```

Once the firmware is flashed, the STM32 runs independently. GDB and OpenOCD do not need to remain connected. With

```
1 openocd \
2   -f board/st_nucleo_f4.cfg \
3   -c "program build/intro.elf verify reset exit"
```

The final reset exit means:

- Reset the STM32.
- Allow it to run.
- Exit OpenOCD.

After that, the program remains in Flash and automatically starts whenever you:

- Press the board's reset button.
- Disconnect and reconnect power.
- Turn the board off and back on.